When Does the Knowledge Graph Help?

Adaptive RAG Routing for EHR-Grounded Clinical Question Answering

Domain

AI in Healthcare · Medical NLP · Retrieval-Augmented Generation · Knowledge Graphs

1. Project Overview

Title	When Does the Knowledge Graph Help? Adaptive RAG Routing for EHR-Grounded Clinical Question Answering

Field	Description
Domain	AI in Healthcare · Medical NLP · Retrieval-Augmented Generation · Knowledge Graphs · Evaluation & Safety
Type	Research-oriented M.Tech project
Primary Venues	CHIL, AMIA Annual Symposium, EMNLP BioNLP Workshop
Backup Venues	IEEE EMBC, IEEE BIBM, HEALTHINF
Hardware Target	Single NVIDIA RTX 4050 (6 GB VRAM)
Base Model	Phi-3 Mini (3-8B SLM) with QLoRA fine-tuning
Key Dataset	MIMIC-IV (structured EHR + clinical notes)

2. Motivation & Problem Statement

2.1 The Gap This Research Fills

Large Language Models (LLMs) exhibit strong performance on medical QA and summarization tasks, but face three critical limitations in real clinical settings: hallucinations, lack of patient-specific grounding, and weak integration with structured EHR data.

Retrieval-Augmented Generation (RAG) partially addresses hallucinations by retrieving external documents, yet most medical RAG systems still rely predominantly on unstructured text. Structured EHR fields and explicit medical knowledge graphs remain underutilized.

2.2 Limitations of Existing Systems

Recent systems such as MedRAG, MedGraphRAG, and MediGRAF have begun combining EHR data with knowledge graphs — but they share a critical assumption that more KG/EHR context always helps. This project challenges that assumption directly:

- When the patient's EHR is dense, extra KG retrieval may add noise or redundancy, increasing hallucination risk from irrelevant context.
- When the EHR is sparse, KG retrieval may be crucial to fill knowledge gaps.
- Different retrieval sources carry different latency and compute cost profiles — relevant for real-world deployment on consumer hardware.

2.3 Central Research Questions

H1	Can an adaptive retrieval router that dynamically chooses between text-only, text+EHR, and text+EHR+KG retrieval reduce clinically relevant hallucinations and latency — while preserving or improving accuracy — compared to always-on hybrid KG-RAG in EHR-grounded clinical QA with small language models?

H2	How does the benefit of KG augmentation depend on EHR sparsity, measured via concrete EHR sparsity metrics (labs, diagnoses, documentation density)?

3. Project Objectives

3.1 Primary Objectives

1. Build an EHR-grounded clinical QA system using a Small Language Model (Phi-3 Mini or comparable 3-8B SLM) running on a single consumer GPU.
2. Design a lightweight healthcare lakehouse integrating MIMIC-IV structured EHR tables, clinical notes, and QA datasets using Parquet + DuckDB.
3. Construct a Medical Knowledge Graph (MKG) for selected internal medicine conditions, with clearly specified ontology sources, construction rules, and manual validation.
4. Implement a hybrid retrieval layer supporting text (vector) retrieval, structured EHR retrieval, and knowledge graph retrieval.
5. Develop an Adaptive Retrieval Router that selects among T (text-only), T+E (text+EHR), and T+E+K (text+EHR+KG) retrieval modes per query.
6. Create a hallucination taxonomy, annotation protocol, and small classifier for EHR-grounded QA.
7. Conduct rigorous experiments with train/val/test splits, oracle and random routing baselines, ablations, and efficiency metrics.

3.2 Secondary Objectives

- Explore EHR-constrained prompting (e.g., blocking contraindicated recommendations based on patient data).
- Release a labeled hallucination evaluation dataset and open-sourced code where MIMIC-IV licensing permits.
- Include clinical face-validity check via expert review of 50 randomly sampled QA outputs.

4. Datasets

4.1 MIMIC-IV

MIMIC-IV (Medical Information Mart for Intensive Care) is the core dataset, providing both structured and unstructured patient data from a tertiary care hospital.

Data Type	Contents
Structured	ICD diagnoses, lab events, vital signs, procedure orders, medication orders, admission and ICU information
Unstructured	Discharge summaries and clinical free-text notes

4.2 QA Data for Fine-Tuning and Evaluation

Fine-Tuning Data (QLoRA)

- MedQA (USMLE) — train split only
- MedQuAD — medical QA pairs from authoritative medical websites
- Synthetic EHR-QA set derived from MIMIC-IV training partition via templates (e.g., 'What is the primary diagnosis?', 'Which lab abnormality suggests X?')

Evaluation Data

- Held-out EHR-QA set from a disjoint MIMIC-IV patient split with manually curated QA pairs
- Optional: MedQA dev/test split for general medical reasoning benchmarking

Data Separation Rule	Fine-tuning, router training, and final evaluation use strictly non-overlapping patient IDs and question sets. This is enforced at the patient level to prevent any form of data leakage.

5. System Architecture

5.1 Healthcare Lakehouse

A lightweight healthcare lakehouse provides unified storage and querying for all structured and unstructured data, enabling fast patient-centric data assembly.

Component	Details
Storage Format	Parquet tables partitioned by domain: patients, admissions, labs, vitals, diagnoses, medications, notes, QA pairs
Query Engine	DuckDB for local analytical queries and joins over Parquet files
Primary Use	Patient-centric joins for EHR snapshots (e.g., last 48h labs + discharge note)
Secondary Uses	Embedding generation, MKG construction (disease-lab co-occurrence), retrieval pipelines, sparsity computation

5.2 Small Language Model (SLM)

- Base model: Phi-3 Mini or comparable 3-8B parameter SLM
- Fine-tuning: QLoRA on MedQA train split, MedQuAD, and synthetic EHR-QA (training partition only)
- Strict separation: no overlap between fine-tuning data and held-out evaluation questions/patients

5.3 Embedding and Vector Retrieval

- Embeddings: BGE-Small and all-MiniLM-L6-v2 for sentence-level representations
- Index: FAISS for fast approximate nearest-neighbor search
- Text retrieval: question -> query encoder -> top-k chunks from clinical notes, MKG textual descriptions, and guideline snippets

5.4 Structured EHR Retrieval

Per-patient EHR snapshots are assembled from the lakehouse, formatted as natural language context:

Example EHR Context	Patient: 65-year-old male. Diagnoses: Type 2 Diabetes, CKD Stage 3. Labs: HbA1c 9.1% (high), Creatinine 2.3 mg/dL (high), eGFR 35 ml/min (reduced). Medications: Metformin 1000 mg BID.

5.5 Knowledge Graph Retrieval

- Entity identification: extract disease codes, drug names, and lab values from question and EHR snapshot

- Subgraph retrieval: 1-2 hop neighborhood around identified entities in Neo4j
- Linearization: subgraph facts converted to natural language statements

Example KG Fact	For CKD Stage 3, ACE inhibitors are recommended as first-line for hypertension unless hyperkalemia is present.

6. Adaptive Retrieval Router

6.1 Routing Modes

Mode	Sources Used	Expected Use Case
T	Text/notes only	General medical definitions, low EHR dependency, dense EHR contexts
T+E	Text + EHR snapshot	Patient-specific questions with available structured records
T+E+K	Text + EHR + KG facts	Sparse EHR, treatment/contraindication questions, comorbid conditions

6.2 Router Input Features

Each question-patient pair is featurized with the following signals:

- Question type (diagnosis / test / treatment / explanation / definition) from a small classifier
- Presence of lab or medication keywords in the question
- EHR sparsity bucket (low / medium / high) derived from the sparsity formula
- KG coverage score: number of relevant MKG nodes/edges found for this encounter
- Retrieval statistics: max similarity score and entropy across top-k retrieved chunks

6.3 Training Protocol (Leakage-Safe)

Three strictly disjoint data splits are used:

Split	Size	Purpose
Router Training	~200 questions	Generate pseudo-labels; train router classifier
Router Validation	~100 questions	Tune router hyperparameters
Held-Out Evaluation	200-300 questions	**Final performance comparison ONLY. Never used for router training or label generation.**

7. Medical Knowledge Graph (MKG)

7.1 Scope and Ontology Sources

- Ontology: Subset of UMLS or SNOMED-CT / Disease Ontology, focused on internal medicine
- Seed diseases: 20-50 common MIMIC-IV conditions (diabetes mellitus type 2, hypertension, CKD stages 3-5, heart failure, etc.)
- Seed entities: diseases, symptoms, lab tests, procedures, and drugs relevant to seed conditions

7.2 Edge Types

Edge Type	Example
disease → symptom	Diabetes mellitus → polyuria
disease → lab_test	CKD → serum creatinine
disease → first_line_treatment	Hypertension + CKD → ACE inhibitor
disease → contraindicated_treatment	CKD + hyperkalemia → ACE inhibitor (contraindicated)

7.3 Construction Process

Ontology-Based Edges

Extract disease-symptom, disease-test, and disease-drug relationships from the chosen ontology (UMLS/SNOMED-CT) or curated clinical guidelines.

EHR Co-occurrence Edges

From MIMIC-IV, compute disease-lab/test co-occurrence frequencies. Add an edge if co-occurrence appears in >5% of admissions for that disease, or exceeds a defined odds ratio threshold.

Manual Validation

Randomly sample 50 edges; validate each against a trusted medical reference. Spurious or incorrect edges are removed or corrected. The validation rate is reported in the paper.

7.4 Implementation and Expected Scale

- Graph database: Neo4j with Cypher queries for subgraph retrieval
- Expected nodes: 1,000 - 3,000 (diseases, symptoms, labs, drugs)
- Expected edges: 5,000 - 10,000
- Reporting: node/edge distribution per disease family

8. EHR Sparsity Definition (H2 Operationalization)

To study H2, EHR sparsity is defined via three quantifiable metrics per patient-encounter:

Metric	Definition	High Sparsity Threshold
n_labs	Number of distinct lab tests recorded	n_labs < 5
n_diag	Number of distinct ICD diagnosis codes	n_diag < 3
d_note	Days since last clinical note before question time	d_note > 7 days

A composite sparsity score S = alpha_1 * 1(n_labs < tau_labs) + alpha_2 * 1(n_diag < tau_diag) + alpha_3 * 1(d_note > tau_days) is computed per encounter. Thresholds and weights are calibrated empirically. Encounters are bucketed into High, Medium, and Low sparsity categories.

H2 Test	Plots will show accuracy and hallucination rate per system (T, T+E, T+E+K, Router) stratified by sparsity bucket. H2 is confirmed if T+E+K (or Router) shows its largest advantage over T/T+E in the high-sparsity bucket.

9. Hallucination Taxonomy, Annotation & Evaluation

9.1 Taxonomy

Label	Definition
Faithful	Answer fully supported by EHR, MKG, and retrieved text; no clinically relevant errors
Partially Hallucinated	Mixture of correct and hallucinated statements; some clinically relevant errors
Hallucinated	Contains at least one serious hallucination of clinical significance

9.2 Hallucination Categories

- EHR-contradicting: conflicts with structured patient data (e.g., states 'no kidney disease' when labs show elevated creatinine)
- KG-contradicting / Guideline-violating: conflicts with MKG or clinical guideline relationships (e.g., recommends a clearly contraindicated drug)
- Unsupported: specific medical claims not derivable from EHR, MKG, or retrieved context

9.3 Annotation Protocol

8. Sample 300-500 questions from the held-out evaluation set, stratified by sparsity bucket and question type
9. For each system output (T, T+E, T+E+K, Router): assign Faithful/Partial/Hallucinated label; if hallucinated, assign a category
10. Use at least two independent annotators; compute Cohen's kappa for inter-annotator agreement on hallucination detection and type
11. Resolve disagreements via structured discussion or adjudication by a third annotator

9.4 Clinical Face-Validity Check

A medically trained reviewer (doctor, resident, or senior medical student) independently evaluates 50 randomly selected QA pairs across all four systems using a 3-point scale: Correct, Partially Correct, or Incorrect/Unsafe. These ratings are reported as clinical face-validity metrics alongside automated evaluation results.

10. Experimental Setup

10.1 Systems Compared

System	Description
T	Text-only RAG (notes and guidelines only)
T+E	Text + structured EHR snapshot
T+E+K	Always-on hybrid: Text + EHR + MKG (baseline for comparison)
Router (proposed)	Adaptive RAG — learned routing among T, T+E, T+E+K per query
Random Routing	Random choice among the three modes (lower bound)
Oracle Routing	Retrospective best mode per question (non-deployable upper bound)

10.2 Evaluation Metrics

Text Quality and Accuracy

- BLEU, ROUGE-L, METEOR against reference answers
- BERTScore (semantic similarity to reference)

Hallucination

- Overall hallucination rate per system
- Per-category rates: EHR-contradicting, KG-contradicting, unsupported
- Refusal/abstention rate

Efficiency

- Average latency per query (milliseconds)
- VRAM usage per query

- Average number of retrieved tokens per mode

10.3 Ablation Studies

- Remove EHR from T+E+K to isolate KG contribution
- Remove KG from T+E+K to isolate EHR contribution
- Analyze performance vs sparsity bucket (H2)
- Analyze router decisions by question type
- Latency vs accuracy trade-off curve (Router vs Always-Hybrid)

11. Technology Stack

Component	Technology
Language Model	Phi-3 Mini (3-8B) with QLoRA (PEFT library)
Embeddings	BGE-Small, all-MiniLM-L6-v2 (HuggingFace)
Vector Index	FAISS
Knowledge Graph	Neo4j (Cypher queries)
Lakehouse Storage	Apache Parquet
Query Engine	DuckDB
ML Framework	PyTorch, HuggingFace Transformers
Orchestration	LangChain / LlamaIndex
Hardware	Single NVIDIA RTX 4050 laptop GPU (~6 GB VRAM)

12. Expected Contributions

#	Contribution	Description
1	**Adaptive RAG Router**	Learned routing mechanism among T, T+E, T+E+K modes for EHR-grounded QA with small LMs; evaluated against random and oracle baselines
2	**Empirical Analysis of KG Benefit**	Quantitative answer to 'when does KG help?' across EHR sparsity levels and question types
3	**Lightweight Lakehouse + MKG**	Clearly specified ontology sources, construction rules, and manual validation; reproducible on consumer hardware
4	**Hallucination Taxonomy & Protocol**	Three-class hallucination taxonomy specific to EHR+KG QA, with annotation protocol, inter-annotator agreement, and small classifier

5	**Open Evaluation Set**	Labeled hallucination dataset and open-sourced code for future research on safe medical RAG with small language models

13. Safety, Ethics, and Limitations

13.1 Ethical Safeguards

- Research prototype only: system is not intended for, and must not be used in, live clinical decision-making
- All experiments use de-identified MIMIC-IV data under appropriate PhysioNet Data Use Agreements
- Hallucination annotations and classifier provide structured research evaluation and are not a substitute for clinical judgment

13.2 Known Limitations (to be documented in paper)

- MKG scope limited to ~20-50 internal medicine conditions; results may not generalize to other specialties
- Router trained on ~200 questions; performance on rare question types may be limited
- Single hardware platform (RTX 4050); scaling behavior on other GPUs not characterized
- MIMIC-IV reflects one hospital system; demographic and institutional biases may affect generalizability

Potential publication **Target Venues**

CHIL · AMIA · EMNLP BioNLP · IEEE EMBC / BIBM

14. Suggested Implementation Timeline

Phase	Task	Deliverables
Phase 1	Data access, lakehouse setup, EHR snapshot pipeline	Working Parquet lakehouse; patient snapshot queries via DuckDB
Phase 2	MKG construction and validation; SLM fine-tuning	Neo4j graph with validated edges; fine-tuned Phi-3 Mini checkpoint
Phase 3	Hybrid retrieval pipeline; synthetic QA generation	T, T+E, T+E+K pipelines running end-to-end; QA datasets ready
Phase 4	Router training; hallucination annotation	Trained router; annotated evaluation set with kappa scores
Phase 5	Final experiments, ablations, clinical face-validity check, paper writing	Complete results; conference-ready paper draft